Thinker AI V1 Architecture Spec

Synthesized from Founder Constitution (Extended), Constitution v1.0, and Founding Charter v1.0

June 2026

1. Purpose of This Document

The three founding constitutions describe Thinker AI's **vision and philosophy**: a multi-agent system that understands goals, plans work, debates solutions, and delivers verified outcomes. They do not specify **how to build it**.

This document translates that philosophy into a concrete, buildable Version 1 (V1) architecture. It defines every agent, the data that flows between them, the technology choices, and the order of implementation — scoped deliberately small so V1 can actually ship.

Guiding constraint for this document: all nine agent roles named across the three constitutions are preserved, but each is implemented with the minimum logic needed to fulfill its role. Complexity is added

in later versions, not V1. Three additional agents are introduced in this spec: Design Agent (Section 5.10) for image generation, Strategy Agent (Section 5.12) for business/product thinking before planning, and the expanded Decision Memory system (Section 7.3) — bringing the total to twelve agent roles in V1.

2. V1 Scope Statement

2.1 What V1 Is

A web-based chat application where a user describes a goal (build an app, a website, a game, or a general task), and a fixed pipeline of twelve lightweight agents — coordinated by Thinker Core — clarifies the request, thinks strategically about it, plans it, builds a single deliverable (including basic image/graphic assets where needed), reviews it, and returns one final answer to the user. Thinker AI supports 100+ languages and responds automatically in the user's language throughout every stage of the pipeline (Section 17).

2.2 What V1 Is Not

- Not a fully autonomous multi-day project system (that is a later phase in the long-term roadmap).
- Not a mobile app (V1 ships as a responsive web app; a mobile wrapper is a launch nice-to-have, not a blocker).
- Not running every request through full multi-model debate — multi-model consensus is invoked selectively (see Section 6.4), not on every single message, to protect cost.
- Not retaining credentials, passwords, or payment data in memory.

2.3 V1 Success Criteria

#	Criterion
1	User can chat normally and get direct answers without triggering the full agent pipeline
2	User can request a small project (app/website/game) and the system asks clarifying questions before building
3	All twelve agent roles are present in code, even if some are simple rule-based stubs rather than full LLM calls
4	Every final deliverable passes through Reviewer before being shown to the user
5	Multi-model consensus works for at least one agent (Judge) end-to-end
6	Project memory persists within a session and reloads on return
7	For projects with a visual asset, the user explicitly approves the final image before the project is marked complete
8	Smart Clarification correctly detects vague goals and activates 3-Level Deep mode (Section 5.2.3)
9	Signature Question fires on every project-type request before Builder runs, and user response is logged (Section 5.2.5)

3. Core Philosophy → Implementation Mapping

Constitution Principle	V1 Implementation
Understand first, plan second, execute third, verify always	Fixed pipeline order, enforced in code, not optional
Requirement Clarity Law — ask minimum questions	Clarification Agent has a hard gate; Builder cannot run

before large work	until requirements are marked complete
Token Economy / Resource Protection Law	Cheap single-model calls by default; multi-model consensus only for high-stakes decisions (Judge approval)
One Goal. One Team. One Final Answer.	Thinker Core merges all agent outputs into exactly one response object before returning to the user
No single model/agent/opinion is final — verified consensus is final	Judge Agent's approval is required before Thinker Core marks a deliverable "final"

The Founder Law

> **Thinker AI must not immediately build what the user asks for. It must first determine whether the user's goal, problem, and assumptions are valid.**

This is the single most important rule that separates Thinker AI from every other AI builder tool. A user who says "build me a Facebook clone" does not necessarily need a Facebook clone — they need their underlying goal addressed, and that goal might be better served by something much smaller or completely different. Building first and validating later wastes the user's time, money, and trust.

This law is enforced architecturally: the pipeline order makes it structurally impossible to reach Builder without first passing through Clarification Agent (which validates requirements) and Strategy Agent (which validates the goal itself). No code is ever written before the thinking is done.

The Preview Before Build Law

> **Thinker AI must show the user what it intends to build before generating the final deliverable. User approval is required at each stage before the next stage begins.**

A ZIP file that surprises the user is a failure, even if the code is correct. The user's mental model of what is being built must match what Thinker AI is actually building — and the only way to ensure this is to show the plan, the blueprint, and the architecture before a single line of final code is written.

This law reduces wasted tokens (Resource Protection Law), reduces user frustration, and makes Thinker AI feel less like a black box and more like a professional development team that keeps the client informed at every stage. See Section 16 for the full 5-stage output pipeline that implements this law.

4. System Architecture

4.1 Pipeline Overview

The pipeline runs in a fixed order, owned by Thinker Core:

1. **Thinker Core** receives the user message.
2. **Intent Agent** classifies it: chat, app request, website request, game request, or task request.
3. **Clarification Agent** checks if requirements are complete; if not, the pipeline halts and returns questions to the user.
4. **Strategy Agent** (project requests only) produces a strategic brief — goal clarity, MVP scope, key risks, success criteria — before any technical planning begins (see Section 5.12). Skipped entirely for simple chat.
5. **Planner Agent** breaks the clarified goal into ordered steps, using the Strategy Agent's brief as context, marking each step's output type (code/text vs. image).
6. **Research Agent** optionally gathers external information for specific steps.
7. **Builder Agent** generates code/text deliverables; **Design Agent** generates image deliverables —

Thinker Core routes each step to whichever agent matches its output type (see Section 4.3).
8. **Reviewer Agent** checks logic, completeness, and basic security for code/text; for images, applies the relevance/appropriateness check from Section 5.10.
9. **Critic Agent** independently looks for weaknesses Reviewer may have missed.
10. **Judge Agent** scores the deliverable and decides approval, optionally invoking Consensus.
11. **Consensus Agent** runs a multi-model vote only when Judge's decision is borderline.
12. **Thinker Core** merges everything into one final response and returns it to the user.

4.2 Thinker Core Responsibilities

Thinker Core is not itself an LLM call — it is the **orchestration layer** (plain backend logic):

- Owns the pipeline state machine for each conversation
- Decides whether a message needs the full pipeline or just a direct chat reply
- Routes each plan step to Builder or Design Agent based on output type (Section 4.3)
- Passes structured data between agents (see Section 7)
- Enforces the Clarification gate
- Merges Builder, Design, Reviewer, Critic, and Judge outputs into one final response
- Writes to memory after each completed stage

4.3 Builder vs. Design Agent Routing

A single project can need both — for example, "build me a landing page" might need Builder for the HTML/CSS and Design Agent for a hero image or logo. Routing is decided per step, not per project:

Planner Step Output Type	Routed To
`code`, `text`, `config`	Builder Agent
`image`	Design Agent

If a step's output type is ambiguous, Thinker Core defaults to Builder and lets Builder request an image sub-step from Design Agent if it determines one is genuinely needed — this keeps routing logic simple and avoids a separate classification call for every step.

4.4 Domain Picker — A Context Layer Above the Pipeline

Thinker AI's UI offers a set of named "agents" — Music, DevOps, Security, Coding, and so on — that the user can pick from before sending a message. This picker is **not a second pipeline and not a bypass of the nine-agent system**. The nine agents in Section 5 are the only system that ever produces a deliverable. The picker is a domain hint that flows into that one pipeline, the same way an answer to a Clarification question does — it is input, not an alternate path.

Why this exists: Intent Agent has to guess the domain of a request from the wording alone. When the user already knows their problem is a security problem or a music request, asking them to pick that from a short list is faster and more reliable than making Intent Agent infer it every time, and it costs no extra LLM call.

4.4.1 How a Domain Selection Changes Pipeline Behavior

Selecting a domain does not skip any of the nine agents — it adds a piece of context that several of them read:

Agent	How the domain hint is used
Intent Agent	Skipped entirely when a domain is already selected — there is nothing left to classify, since the user told the system what kind of work this is. The intent type used everywhere else is derived directly from the domain (see Section 4.4.2).

Clarification Agent	Uses domain-specific required fields where they exist (e.g. a Music domain might require genre/mood instead of the generic checklist in Section 5.2).
Planner Agent	Receives the domain name as part of its prompt, so step descriptions are framed appropriately (e.g. “implement OAuth2 flow” rather than a generic “implement authentication” for a Security-domain request).
Builder Agent	Receives the domain as additional system-prompt context, biasing it toward domain-appropriate conventions, libraries, and tone.
Reviewer / Critic / Judge	Unaffected — quality, security, and completeness checks apply the same way regardless of domain.

4.4.2 Three Cases

Case 1 — User selects a domain, then describes the request. The domain name is attached to the request as soon as it reaches Thinker Core. Intent Agent is skipped; the domain maps directly to one of the existing intent types plus a domain label (e.g. “Music” maps to `intent: task`, `domain: music`). The rest of the pipeline runs exactly as described in Section 4.1, with every agent that reads `domain` (per the table above) doing so.

Case 2 — User describes the request without selecting a domain. Intent Agent runs as normal (Section 5.1), but its classification prompt is extended to also output a best-guess `domain` label from the same list the picker shows, so Planner and Builder still get a domain hint even though the user didn’t pick one explicitly. This costs nothing extra — it’s the same Intent Agent call, just returning one more field.

Case 3 — CEO Agent (the default/no-domain option). CEO Agent is not a domain in the same sense as the other fifteen — selecting it (or sending no domain at all) means “use your own general judgment,” and the pipeline behaves exactly as it does today: Intent Agent classifies normally, no domain-specific bias is added anywhere downstream. CEO Agent is the default state, not a sixteenth specialization.

4.4.3 Implementation Note (Current Gap)

As of the codebase reviewed alongside this spec, the domain picker exists in the mobile UI (`AgentPanel.tsx`, `lib/agents.ts`) and is fully interactive, but the selected domain is never included in the request sent to `/api/pipeline` — the backend pipeline has no domain-awareness at all. This means the picker currently has no effect on the agents’ behavior, which contradicts the design in this section. Closing this gap requires two changes: the mobile request body must include the selected domain, and `runThinkerCore` (and the agents listed in Section 4.4.1) must accept and use it.

5. Agent Specifications

Each agent is defined with its role, input, output, V1 implementation level, and model tier used.

5.1 Intent Agent

- **Role:** Classify the incoming message as chat, app request, website request, game request, or task request.
- **Input:** Raw user message plus the last five messages of context.
- **Output:** Intent type and a confidence score.
- **V1 Implementation Level:** Single LLM call with a classification prompt. No fine-tuning.
- **Model Tier:** Fast/cheap (Section 6.1).

5.2 Clarification Agent ★ UPDATED

> **This is the most critical agent in the entire pipeline.** If Clarification is weak, the entire

> downstream pipeline fails regardless of how good the other eleven agents are: Wrong Goal → Wrong Strategy → Wrong Plan → Wrong Output. No other agent can fix a wrong goal once it enters the pipeline. This section upgrades Clarification Agent from a Requirement Collector to a true **AI Interviewer + Psychologist + Startup Mentor**.

- **Role:** Enforce the Requirement Clarity Law AND run Smart Clarification — detect emotional context, surface hidden goals, challenge assumptions, and validate the idea before any technical work begins.
- **Input:** Intent type plus the full conversation so far, plus emotional tone signals.
- **Output:** A completeness flag, a list of smart questions, known requirements, assumption flags, and the Signature Question response.
- **V1 Implementation Level:** LLM call against Smart Clarification Layer (Section 5.2.2) + 3-Level Deep Clarification (Section 5.2.3) + Constraint & Assumption Discovery (Section 5.2.4) + Signature Question Protocol (Section 5.2.5).
- **Model Tier:** Fast/cheap — but with enhanced prompt engineering.
- **Gate Rule:** If requirements are not complete, Thinker Core returns the questions to the user and does not proceed to Planner. This is a hard stop, not a suggestion.

5.2.1 Goal Discovery Mode

Most AI tools assume the user knows what they want. Thinker AI does not. When a user's request is too vague to classify or plan (e.g. "I want to build a project," "I want to start a startup," "I want to help people"), Clarification Agent enters Goal Discovery Mode before asking any technical questions.

When Goal Discovery Mode activates: when the user's message contains no specific product type, domain, or problem — only a direction or desire. Examples: "I want to build something," "I have a business idea," "I want to create an app but don't know what."

Five layers of discovery, one question per layer at a time:

Layer	Purpose	Example Question
1. Goal Discovery	Understand what the user actually wants to achieve	"Why do you want to build this? What is the real goal?"
2. Problem Discovery	Identify the real problem being solved	"What specific problem will this project solve? Do you or others experience this problem?"
3. Audience Discovery	Clarify who this is for	"Who will use this? What do they currently use instead?"
4. MVP Discovery	Find the minimum viable starting point	"If you could only launch with 3 features, which 3 would they be?"
5. Success Discovery	Define what success looks like	"How will you know this worked 90 days after launch?"

Rule: Clarification Agent asks only one question at a time in Goal Discovery Mode, waits for the user's answer, then asks the next. This is not a form — it is a conversation. The session ends when enough is known to hand off to Strategy Agent for validation, which runs before any technical planning begins.

Idea Stress Test (within Goal Discovery): if the user's stated goal appears to conflict with a crowded market or is based on a questionable assumption, Clarification Agent asks one pointed stress-test question before proceeding:

- User: "I want to build an app like Facebook" → Thinker AI: "What will your app do that Facebook doesn't? Which specific audience does Facebook fail to serve well?"
- User: "An app for everyone" → Thinker AI: "Who specifically will be your first 100 users?"

The goal of the stress test is not to discourage the user but to surface assumptions early, before Builder writes a single line of code.

5.2.2 Smart Clarification Layer

The Smart Clarification Layer upgrades Clarification Agent from reactive (asks based on what the user said) to predictive (anticipates where the request is likely to fail). Five capabilities are added — all implemented through enhanced prompt engineering on the existing single Clarification call, with no extra LLM calls added.

Emotional Context Detection: Clarification Agent reads tone, not just words. When word choice signals stress, urgency, or frustration, the agent softens its language, avoids aggressive challenge questions, and leads with empathy before discovery. Tone detection runs at the same time as intent classification — no extra LLM call needed.

Contradiction Detection: When a user's later requests contradict an earlier stated constraint, Clarification Agent catches the conflict and surfaces it explicitly, asking the user to clarify which direction takes priority. The system never silently resolves contradictions by building the larger or more expensive interpretation.

Assumption Surfacing: Users frequently assume things they have not explicitly stated. When a request implies multiple hidden components, Clarification Agent surfaces each unstated assumption and confirms them with the user before Planner locks in the scope. Nothing is assumed silently.

Progressive Disclosure: Questions adapt based on previous answers. When an earlier answer makes a follow-up question irrelevant, that question is automatically skipped. Every question must be conditioned on what is actually unknown given the answers so far — never from a static checklist that ignores what has already been learned.

Intent Confidence Threshold: Clarification Agent outputs an internal confidence score for how well-understood the goal is.

Confidence	Action
90%+	Proceed directly to Strategy Agent
70–90%	Ask one confirmation question, then proceed
Below 70%	Activate Goal Discovery Mode (Section 5.2.1)

This threshold is never shown to the user — it is internal routing logic only.

5.2.3 3-Level Deep Clarification

Standard clarification operates on one level — what the user wants. Smart Clarification operates on three levels simultaneously, peeling back the layers of a request to reach its true core.

Level	Focus	What the Agent Does
Level 1 — Surface Goal	What did the user literally say?	Captures the stated goal without judgment. Confirms understanding before probing deeper.
Level 2 — Hidden Goal	Why do they want this? What is the underlying motivation?	Asks why the goal matters. Surfaces the real outcome the user is trying to achieve, which may be different from what they literally asked for.
Level 3 — Reality Check	Are their assumptions realistic?	Identifies gaps between the stated goal and current situation. Presents the reality gap as a question, never a judgment.

Rule: Level 3 always challenges, never discourages. The agent presents the reality gap as a question ("are you looking for X or Y?") rather than a judgment ("that's impossible"). The user's ambition is respected; the agent's job is to make sure it is informed ambition.

5.2.4 Constraint & Assumption Discovery

These two discovery layers are triggered conditionally — only when the goal is large or complex enough to make them relevant. They do not run on every request.

Constraint Discovery surfaces the real-world limits the user is operating within. They are almost never stated voluntarily but profoundly affect what can actually be built:

Constraint	Question Asked
Time	How much time is available? What is the target launch date?
Budget	What is the budget? Are you the developer, or hiring one?
Skill	What is your technical skill level? Will you be writing code?
Network	Do you have access to real users who have this problem?
Existing assets	Is there existing code, design, or data to build on?

Assumption Discovery surfaces the hidden assumptions that, if wrong, would make the entire project fail:

User Statement	Hidden Assumption	Clarification Question
“Build a social app”	Users will switch from existing platforms	What evidence suggests users will adopt this over existing alternatives?
“Build a food delivery app”	Needs full rider + restaurant + customer system	Could the same problem be solved with a simpler tool before building a full platform?
“Automate everything with AI”	AI can reliably perform this task	How reliably can AI handle this task today, and what happens when it fails?

V1 vs V2 layer coverage:

Layer	V1	V2
1. Goal Discovery	Always	Always
2. Problem Discovery	Always	Always
3. Audience Discovery	Always	Always
4. Constraint Discovery	Conditional	Always
5. Assumption Discovery	Conditional	Always
6. Alternative Discovery	Conditional	Always
7. MVP Discovery	Always	Always
8. Success Discovery	Always	Always

5.2.5 Signature Question Protocol

Every project-type request — without exception — receives exactly one Signature Question before Builder is ever called. This is Thinker AI’s defining differentiator from every other AI builder tool.

The Signature Question:

> “Why do you believe this is the right solution?”

This single question stops more wrong projects than any amount of technical review. A user who cannot answer it has not yet validated their own solution — and Thinker AI should not build it until they can.

Why this question works: - It forces the user to articulate their reasoning, not just their desire - It surfaces assumptions the user has not consciously examined - It opens space for Strategy Agent to

suggest a better path if the current one is wrong - It costs nothing — no extra LLM call, just one question before Planner runs - It is asked in the user's language (Section 17)

Response routing:

User Response Type	System Action
Clear, confident answer with reasoning	Proceed to Strategy Agent with the reasoning as additional context
Vague answer with no supporting reasoning	Trigger Assumption Discovery (Section 5.2.4) before proceeding
User declines and asks to proceed immediately	Acknowledge, note the risk explicitly in the Strategy Brief, and proceed — never block the user
Answer reveals a better solution exists	Strategy Agent flags the alternative before Planner runs

Hard rule: Clarification Agent never blocks a user who insists on proceeding. The Signature Question is a thinking tool, not a gatekeeping mechanism. If the user says "just build it," Thinker Core logs the unanswered question, notes the risk in the Strategy Brief, and proceeds. The user's autonomy is always respected.

5.3 Planner Agent

- **Role:** Convert a clarified goal into an ordered list of concrete steps.
- **Input:** The known requirements object + Strategy Agent brief + Signature Question response + Constraint findings from Clarification Agent.
- **Output:** A list of steps, each with an id, description, output type, and dependencies.
- **V1 Implementation Level:** Single LLM call producing a structured step list. No re-planning loop in V1 (re-planning on failure is a later feature).
- **Model Tier:** Mid-tier reasoning.

5.4 Research Agent

- **Role:** Gather external information when a step requires it (for example, "what does a modern fintech landing page look like").
- **Input:** A single step from the Planner's list, flagged as needing research.
- **Output:** Findings and sources.
- **V1 Implementation Level:** Optional — only invoked if Planner flags a step. Uses web search. Skipped entirely for purely generative steps, since most code and UI generation does not need it.
- **Model Tier:** Fast/cheap, with search tool access.

5.5 Builder Agent

- **Role:** Produce the actual deliverable — code, copy, configuration, or content — for each Planner step.
- **Input:** Step description, any research findings, and relevant project memory.
- **Output:** Artifact type, content, and any generated files.
- **V1 Implementation Level:** The most token-heavy agent. Runs once per step, sequentially, in V1 (parallel step execution is a later optimization).
- **Model Tier:** Strongest available coding/reasoning model.

5.6 Reviewer Agent

- **Role:** Run the quality control checklist: accuracy, logic, security, completeness, and user-goal alignment.
- **Input:** Builder's output plus the original requirements.
- **Output:** A pass/fail flag and a list of issues with severity.
- **V1 Implementation Level:** Single LLM pass against the five-point checklist as explicit criteria. If it

fails with high-severity issues, the request goes back to Builder once — capped at one retry in V1 to control cost.
- **Model Tier:** Mid-tier reasoning.

5.7 Critic Agent

- **Role:** Independently look for weaknesses that Reviewer's checklist-based pass might miss — edge cases, unclear UX, fragile assumptions.
- **Input:** Builder's output plus Reviewer's findings.
- **Output:** A list of concerns with an overall severity rating.
- **V1 Implementation Level:** Single LLM call, deliberately given a different prompt persona than Reviewer (adversarial/skeptical framing) so it doesn't just repeat Reviewer's findings.
- **Model Tier:** Mid-tier reasoning, same tier as Reviewer but a different prompt.

5.8 Judge Agent

- **Role:** Score the deliverable on accuracy, feasibility, safety, cost, and completeness, then decide final approval.
- **Input:** Builder output, Reviewer result, and Critic concerns.
- **Output:** A score per criterion and an approval decision.
- **V1 Implementation Level:** If the combined score is clearly high or clearly low, Judge decides alone. If borderline (within a defined threshold of the pass line), Judge invokes the Consensus Agent instead of deciding unilaterally.
- **Model Tier:** Strongest available reasoning model.

5.9 Consensus Agent

- **Role:** Implement the debate-system / multi-model consensus described across all three constitutions, used only when Judge needs a tie-breaker.
- **Input:** The same question Judge was uncertain about.
- **Output:** Each model's vote and reasoning, plus a final verdict.
- **V1 Implementation Level:** Sends the identical question to three different models — one from each of three providers — collects their verdicts, and takes the majority vote. Ties default to the more conservative outcome (reject or ask the user).
- **Model Tier:** Three different providers, called in parallel.

5.10 Design Agent

- **Role:** Handle visual/graphic output — logos, illustrations, UI mockup imagery, posters, icons — that text-generation models cannot produce directly. This agent is a tenth role added beyond the original nine named in the constitutions, scoped specifically to image generation.
- **Input:** A Planner step flagged as `output_type: image`, plus any style/brand requirements gathered during Clarification.
- **Output:** One or more generated image files plus a short text description of what was generated, stored as artifacts alongside Builder's output.
- **V1 Implementation Level:** A single call to an external image-generation API (not a code/text LLM call). Does not attempt 3D models or animation/video — see Section 13 for what's explicitly excluded. Output still passes through Reviewer for a basic relevance/appropriateness check, but the five-point code checklist in Section 5.6 does not apply to images; Reviewer instead checks the image matches the request and contains no inappropriate content.
- **Model Tier:** External image-generation provider (e.g., DALL-E or Stability AI), called only from the backend — same credential-handling rule as every other model call (Section 9).

5.11 Visual Asset Clarification & Approval Flow

Code can be checked against an objective checklist — logic, security, completeness. A logo or illustration cannot: "is this a good logo" is a matter of taste, not correctness. Reviewer, Critic, and Judge are not

equipped to make that call on the user's behalf, so visual assets get a different path through the pipeline than code does.

Step 1 — Style input, asked once by Clarification Agent. When Planner marks a step as `output_type: image`, Clarification Agent asks a single style question before that step reaches Design Agent: does the user want to describe the look themselves (colors, mood, references), or leave it entirely to Design Agent's judgment? This is one extra question, consistent with the Requirement Clarity Law's "ask the minimum necessary" principle — not a long design brief.

- If the user describes a style, those details are stored in `known_requirements` and passed directly into Design Agent's prompt.
- If the user says to decide freely, Design Agent proceeds without constraint, using only the project's general context (e.g. app name, purpose, target audience already gathered earlier in Clarification).

Step 2 — Generated image returns to the user for a decision, not just to Reviewer. Unlike code, a generated image is shown to the user as part of the final response with three options: **approve**, **reject and regenerate**, or **revise** (describe what to change). This is a deliberate exception to "One Goal. One Team. One Final Answer." for this asset type specifically — the final answer for a code deliverable is the agents' consensus, but the final answer for a visual deliverable is the user's own approval, because that is the only valid judge of taste.

- **Approve:** the image is finalized and stored in project memory as-is.
- **Reject and regenerate:** Design Agent runs again with the same brief, producing a different result.
- **Revise:** the user's feedback (e.g. "make it more minimal," "different color") is appended to the original brief and Design Agent runs again with the updated input.

Step 3 — Retry limit. To protect the Resource Protection Law, regeneration is capped at 3 attempts per visual asset in V1. If the user is still unsatisfied after 3 attempts, Thinker Core surfaces this directly rather than silently continuing to regenerate, and asks whether to proceed with the current best version or describe the requirement differently from scratch.

5.12 Strategy Agent

- **Role:** Think about the *why* and *how to succeed* behind a user's goal — not the technical execution steps (that is Planner's job), but the business, product, and market thinking that determines whether the thing being built will actually work in the real world. This is the agent that makes Thinker AI more than a code generator.
- **When it runs:** Only for project-type requests (`app_request`, `website_request`, `game_request`, `task_request`) — not for simple chat. It runs **after Clarification and before Planner**, so the strategic context it produces becomes part of Planner's input.
- **Input:** The clarified requirements from Clarification Agent, plus the domain hint (if selected via the Domain Picker, Section 4.4), plus the Signature Question response and Constraint findings from Clarification Agent (Sections 5.2.4–5.2.5).
- **Output:** A structured strategy brief containing:
  - **Goal clarity:** What the user is actually trying to achieve (reworded from requirements to outcomes, e.g. "get 100 daily orders in Dhaka" not "build a food delivery app")
  - **MVP scope:** The minimum version of this that would prove the idea works, versus what can wait
  - **Key risks:** What is most likely to go wrong (technical, market, or cost risks)
  - **Success criteria:** How the user will know if this worked, stated as measurable outcomes
- **V1 Implementation Level:** Single LLM call with a structured prompt that explicitly asks for business/product thinking, not technical steps. The output is a short brief (not a long document), passed to Planner as context alongside the requirements.
- **Model Tier:** Mid-tier reasoning — same tier as Planner, since this is structured reasoning, not code generation.

What Strategy Agent does NOT do: it does not replace the user's judgment on business decisions. It surfaces questions and risks; the user (via Clarification Agent's follow-up if needed) makes the final call. Strategy Agent's brief is a thinking aid, not a business plan that overrides what the user asked for.

6. Model & Cost Strategy

6.1 Model Tiers

Tier	Used By	Purpose
Fast/cheap	Intent, Clarification, Research	High-frequency, low-complexity decisions
Mid-tier reasoning	Planner, Strategy, Reviewer, Critic	Structured reasoning, moderate complexity
Strongest available	Builder, Judge	Highest-stakes generation and final scoring
Image generation (single provider)	Design Agent	Visual asset generation, not a reasoning task
Multi-model (3 providers)	Consensus	Only invoked on borderline Judge decisions

6.2 Resource Protection Law in Practice

- Every pipeline run is logged with token cost per agent.
- The Clarification gate prevents Builder, the most expensive agent, from ever running against an unclear request.
- The Consensus Agent, roughly three times the cost of a single call, is invoked only when Judge's confidence is borderline — estimated at well under 20% of all requests.
- The Reviewer retry loop is capped at one retry to prevent runaway cost on a stuck request.

6.3 Direct Chat Path — Dual-Model Verified

Not every message needs the full agent pipeline. If Intent Agent classifies a message as ordinary chat — a general question, a follow-up, or a clarification reply that doesn't start a new project — Thinker Core skips the full agent pipeline (Planner, Builder, Reviewer, Critic, Judge are not invoked). The full pipeline is reserved for app, website, game, and task requests.

Even on this fast path, simple chat is never answered by a single model alone. Every direct-chat message is sent to **two different models in parallel**, and Thinker Core compares the two responses before answering:

- **If the two responses agree** (same facts, same conclusion, no material contradiction): Thinker Core merges them into one clean answer and returns it. The user never sees that two models were involved — only the Verified output (Section 6.5, Model Identity Concealment, still applies in full).
- **If the two responses disagree:** Thinker Core does not guess which one is right. It either
    a. presents the answer with an explicit note that there is some uncertainty on this point, or (b) escalates to a third model as a tie-breaker if the disagreement is on a factual claim that matters (numbers, dates, technical facts) — this escalation follows the same majority- vote logic as Consensus Agent (Section 5.9), just triggered from the chat path instead of Judge.

This is a deliberate cost-for-trust tradeoff. Every simple chat message now costs roughly twice what a single-model answer would cost — this is treated as the baseline cost of the product, not an optional upgrade. The founding principle is that Thinker AI never gives an unverified answer, not even for a quick question. See Section 18.5 for how this affects the credit cost table.

6.4 When Consensus Actually Fires

Judge Score Result	Action
Clearly passes all five criteria	Approve directly, no Consensus call

Clearly fails (for example, safety or accuracy critically low)	Reject directly, send back to Builder
Borderline on any criterion	Invoke Consensus Agent for that criterion only

6.5 Model Identity Concealment

Thinker AI's value proposition is that the user is talking to one coherent system that has already checked itself, not to a specific underlying model. Which model — and which provider — answered any given step is an implementation detail, never a fact disclosed to the user.

Hard rule: no provider or model name (Claude, GPT, Gemini, or any other) appears anywhere the user can see — not in chat responses, not in the Consensus Agent's internal "votes," not in error messages, and not in any debug/admin view shipped in V1. Internally, each model slot is referred to by a generic label (e.g. "Agent A," "Agent B," "Agent C") in code, prompts, and logs alike — the rule applies to the codebase's own naming, not only what's rendered in the UI. This is stricter than typical practice (most products show model names in at least a debug panel) and is treated as a product requirement, not just a UI cosmetic choice.

What this changes from the Consensus Agent design in Section 5.9: the three persona names already used there ("Conservative Reviewer," "Pragmatic Engineer," "Critical Skeptic") satisfy this rule as written — they describe a reasoning style, not a vendor. The constraint is on what gets added later: if Consensus is extended to use more models (see Section 6.6), every new slot must follow the same generic-label convention, never the provider's brand name.

Failure-path implication: if a model call fails (timeout, rate limit, outage), the error shown to the user must describe the failure generically (e.g. "one of the verification steps didn't respond — retrying") rather than naming which provider failed. This applies even in server logs intended for the founder's own debugging — the convention should be consistent so there's no accidental leak through a shared log file or support ticket screenshot.

6.6 Model Pool Size — V1 vs. Long-Term

The constitutions describe Thinker AI's long-term identity as combining "many minds" into one intelligence layer, which implies the model pool grows over time, not staying fixed at three. V1 deliberately keeps this small for cost and complexity reasons (Section 6.2), but the architecture should not hard-code "three" anywhere it doesn't have to:

- **V1:** exactly three model slots, as specified in Section 5.9 — this is a deliberate scope limit, not a ceiling on the design.
- **Beyond V1:** the model pool becomes configurable — Consensus (and potentially other agents) can draw from a larger pool of available models, with the founder able to add or remove a provider without a pipeline redesign. This is noted here as a known direction so V1's code isn't built in a way that makes it hard to do later (e.g. avoid hard-coding "3" as a magic number in places where "however many slots are configured" would cost nothing extra now).

7. Data Flow & State

7.1 Pipeline State Object

Conceptually, each project in progress is tracked as a single state record containing: an id, the user id, the classified intent type, the requirements object and whether it's complete, any outstanding clarification questions, the plan's steps, any research findings, the Builder's output artifacts, the Reviewer result, the Critic's concerns, the Judge result, the Consensus result (if invoked), a status field (clarifying, planning, building, reviewing, judging, complete, or failed), and created/updated timestamps.

The following fields are added by the Smart Clarification Layer (Section 5.2.2–5.2.5):
`signature_question_response` (the user's answer to the Signature Question, or null if declined),

`constraint_findings` (output of Constraint Discovery — time, budget, skill, network, existing assets), and `assumption_flags` (list of surfaced assumptions and whether each was confirmed or flagged as uncertain by the user). These three fields are passed to Strategy Agent and Planner Agent as additional context.

For projects with visual assets, the state also tracks, per image: the current attempt number (capped at 3 per Section 5.11), the style input source (user-described or agent-decided), and an approval status (pending, approved, revising). The overall project cannot move to `complete` while any image's approval status is still `pending` or `revising`.

For post-delivery fix tracking (Section 9.5), the state also holds: a `medium_fix_count` (resets to 0 at project start, increments on each Builder+Reviewer+Critic cycle), a `full_rebuild_count` (increments each time the pipeline runs from Planner onwards), and a `version_number` (increments each time Builder saves a new output). The version history itself is stored as a separate collection of up to 10 records per project, each containing the full Builder output and a timestamp, keyed by `project_id` and `version_number`.

7.2 Memory System

Memory Type	Scope	V1 Storage	Retention Rule
Short-term	Single conversation turn	In-memory / session	Cleared on session end
Project memory	One project across its lifecycle	Database, keyed by project id	Persists until the user deletes the project
Long-term preference	User-level, across projects	Database, keyed by user id	Persists indefinitely unless the user clears it
Decision memory	User-level, explicit saved decisions	Database, keyed by user id	Persists indefinitely unless the user revokes a decision

Hard rule carried over unchanged from all three constitutions: passwords, PINs, API keys, and other credentials are never written to any memory layer, even temporarily.

7.3 Decision Memory — Detail

Decision Memory is the fourth memory type and deserves its own section because it works differently from Long-term preference.

Long-term preference is learned passively — Thinker AI infers that a user prefers a certain style based on patterns across past projects. The user doesn't explicitly set it; the system gradually builds a picture of their habits.

Decision Memory is set explicitly — the user makes a deliberate statement like "always use React for my projects" or "never use MongoDB" and expects the system to follow that rule every time without asking again. This is not a preference to be weighed; it is a standing instruction to be followed.

How it works in V1:

- When a user states a rule explicitly (Clarification Agent detects the phrasing "always," "never," "from now on," "for all my projects," or similar), Thinker Core writes it to Decision Memory with a confirmation to the user: "Saved — I'll remember to always use React for your projects. You can change this any time."
- On every subsequent project, relevant decisions are pulled from Decision Memory and injected into Planner Agent's context automatically — the user is never asked again unless they say something that contradicts a saved decision.
- If a new request conflicts with a saved decision, Thinker Core surfaces the conflict explicitly: "You asked for Vue.js here, but your saved preference is React. Which should I use for this project?" — and lets the user choose, rather than silently overriding.

What does NOT go into Decision Memory: vague statements like "make it look nice" or "make it fast" — these are not actionable rules, they are quality expectations that Reviewer and Critic already handle. Decision Memory is only for explicit, specific, repeatable instructions.

8. Agent Fallibility & Error Containment

8.1 Why This Section Exists

Every agent in this pipeline is an LLM call, and every LLM call can be wrong. Section 5 defines what each agent is supposed to do; this section defines what happens when an agent does it incorrectly — including when the agent that is supposed to catch a mistake is itself the one making it.

Three distinct failure modes are addressed:

1. **A single agent gives a wrong verdict.** For example, Reviewer marks a flawed deliverable as passing, because Reviewer is a fallible LLM call, not a guaranteed-correct checker.
2. **A wrong verdict propagates silently.** Once one agent has said “this passed,” downstream agents that see that verdict in their context may anchor on it and look less critically at the same issue, letting the original mistake travel untouched through the rest of the pipeline.
3. **Correlated failure across agents.** If Reviewer, Critic, and Judge are all the same underlying model family, they may share the same blind spot and all miss the same problem independently, which defeats the purpose of having multiple checking agents at all.

8.2 Containment Rules

Rule	What It Prevents
No single agent’s approval is final. Reviewer’s pass does not release a deliverable to the user — Judge approval is always required on top of it.	A single fallible verdict (failure mode 1) reaching the user unchecked.
If Reviewer and Critic disagree on whether an issue is real, Thinker Core automatically routes the disputed point to the Consensus Agent rather than letting Judge break the tie alone.	A disagreement getting silently resolved by whichever agent happens to run last (failure mode 2).
Critic’s prompt explicitly withholds Reviewer’s verdict until after Critic has formed its own opinion, then asks Critic to compare.	Critic anchoring on “Reviewer already said it’s fine” and skipping its own independent check (failure mode 2).
For a defined set of high-risk categories (authentication, payment handling, data deletion, permission/access control), Judge cannot approve solely because Reviewer and Critic both passed — Consensus Agent is invoked automatically regardless of score.	A single shared blind spot across same-family models approving dangerous code unchallenged (failure mode 3).
Every rejection or override (Critic overruling Reviewer, Consensus overruling Judge) is logged with both verdicts and the reasoning, not just the final outcome.	Loss of visibility into how often agents actually disagree, which is needed to tell if this system is working.

8.3 What This Does Not Solve

This containment strategy reduces the chance of a bad deliverable reaching the user undetected — it does not guarantee correctness. Consensus Agent’s majority vote can still be wrong if a mistake is common enough across providers (for example, a genuinely obscure security issue all three models fail to recognize). No automated review pipeline replaces a human inspecting high-stakes output, particularly around authentication, payments, or anything touching real user data. V1 should be treated as catching the common cases, not as a substitute for judgment on anything that matters a lot.

9. Pipeline Observability & Post-Delivery Feedback

9.1 The Silent Failure Problem

When a user says “this output is bad,” the problem could have originated at any point in the pipeline:

- **Intent Agent** misclassified the request (e.g. treated a mobile app request as a web request)
- **Planner Agent** produced wrong or incomplete steps (e.g. missed the payment flow entirely)
- **Builder Agent** wrote incorrect code for a correct plan
- **Reviewer Agent** failed to catch a bug that existed
- **Judge Agent** approved something it should have rejected

Because these agents run sequentially and each feeds the next, a mistake at step 1 will silently propagate through steps 2, 3, 4, and 5 — and the final bad output gives no indication of where the chain broke. Without observability, even the founder cannot debug what went wrong. This is called a **silent failure**, and it is a serious operational risk in any multi-step pipeline.

9.2 Per-Agent Logging (Founder-Visible)

Every agent execution must write a structured log entry — not shown to the user, but accessible to the founder/developer — containing:

Field	What It Records
`agent_name`	Which agent ran (Intent, Planner, Builder, etc.)
`input_summary`	A short description of what this agent received as input
`output_summary`	A short description of what this agent produced
`duration_ms`	How long the agent took in milliseconds
`status`	`success`, `failed`, or `skipped`
`error_detail`	If status is `failed`, the error message — using generic terms, never model/provider names (Section 6.5)
`confidence`	Where applicable — Judge’s score, Consensus vote breakdown, Reviewer pass/fail
`retry_count`	How many times this agent was retried before succeeding or giving up
`clarification_depth`	Which Clarification layers ran (1–5 core, 6–8 smart layers) — Clarification Agent only
`signature_q_answered`	Whether the user answered the Signature Question, declined, or was not asked — Clarification Agent only

This log is written to the same database that holds `ProjectState` (Section 7.1), keyed by `project_id` and `agent_name`. It is never shown in the user-facing UI — it is the founder’s debugging tool, not a product feature.

9.3 Failure Tracing — How to Find Where It Broke

When a user reports bad output, the debugging process using these logs becomes straightforward:

1. Pull the `project_id` for that conversation
2. Read the per-agent log entries in order
3. The first entry where `status: failed` or `confidence` drops sharply is where the chain broke
4. Fix that specific agent’s prompt/logic — not the whole pipeline

Without this, debugging requires guessing. With it, failures become traceable in minutes rather than hours.

9.4 Post-Delivery Feedback Loop

When a user says “this is bad” or “fix this,” the system needs to understand **what kind of bad** before doing anything — because the fix is different depending on the root cause:

User’s Problem	System’s Response
Bug in the output (e.g. login doesn’t work)	Ask user to describe the bug, then send back to Builder with the bug description + original plan. Reviewer re-runs after fix.
Missing feature (e.g. “where is the cart?”)	Treat as a new small requirement. Run Clarification → Planner (for just the missing piece) → Builder → Reviewer. Do not rebuild the whole project.
Style/design not to taste (e.g. “looks ugly”)	Follow the Visual Asset revision flow from Section 5.11 for images; for code/layout, ask for specific style feedback and send back to Builder with that context.
Vague (“it’s bad”, no detail)	Ask one clarifying question: “Is there a specific feature that’s broken, something missing, or does it look wrong?” — then route based on the answer.

Important rule: the system never silently rebuilds the entire project in response to vague negative feedback. Rebuilding everything wastes tokens (Resource Protection Law, Section 6.2) and usually produces a different set of problems, not a fixed version of the original. The fix should be targeted at the specific problem, not a full restart.

9.5 Fix Limits — Cost-Based, Not Count-Based

The original design capped fixes at 3 per session. This is wrong for real-world use: a user working on a large project naturally makes many small adjustments over time, and hitting an arbitrary limit after 3 tweaks creates frustration without protecting much cost. The correct approach is to limit based on **how expensive the fix is**, not how many times the user has asked.

Three tiers of fix, with different limits:

Fix Type	What Runs	Cost	Limit
Small targeted fix (e.g. change a color, update a label)	Builder only	Low	Unlimited — these are cheap and expected
Medium fix (e.g. fix a bug, add a missing feature)	Builder + Reviewer + Critic	Medium	Up to 10 per project session
Full rebuild (user wants to start over from requirements)	Full pipeline from Planner onwards	High	Up to 3 per project session

Rollback (restore a previous version) counts as zero cost — no agent runs, no tokens spent, just a database lookup. Rollbacks never count against any limit.

When a medium or full-rebuild limit is reached, Thinker Core tells the user directly: “We’ve done several major revisions on this project. Would you like to describe your requirements differently from the beginning, or continue refining the current version?” — and waits for a clear answer before doing anything.

9.6 Version History

Every time Builder produces an output — whether for the initial delivery or a subsequent fix — Thinker Core saves that version to the database before overwriting the current state. This means every project has a full history of what it looked like at each stage.

What this enables:

- **Rollback on request:** if a user says “the previous design was better” or “go back to how it was before,” Thinker Core shows the available versions (e.g. “Version 1, Version 2, Version 3 — which would you like?”) and restores the chosen one instantly, with no rebuild.
- **No lost work:** a user who accepts a change and then regrets it is not stuck — they can always go back.
- **Diff context for fixes:** when a user reports a bug after a design change, Builder can compare the current version against the last working version to understand what changed, rather than guessing.

Version storage rule: versions are stored per project, keyed by `project_id` and a sequential `version_number`. Only the last 10 versions are kept per project in V1 — older versions are deleted automatically to manage storage cost. The user is told this limit exists if they try to go back further than 10 versions.

Rollback flow:

1. User says “the previous version was better” or “go back to the previous version”
2. Thinker Core lists available versions with a short description of what changed in each
3. User picks one
4. That version is restored as the new current state — no Builder call, no pipeline
5. Fix cycle counters are not affected — rollback is free

9.7 Selective Rollback & Merge

A full rollback restores everything from a previous version. But a user may want to go back to an older version while keeping specific things from the current one — for example: “the previous design was better, but keep the new login flow and the updated color scheme.” This is a **selective rollback**.

Five rules, always followed:

Rule 1 — Full Rollback: when the user wants everything from a previous version with no exceptions, restore that version exactly as it was. No Builder call needed — database lookup only.

Rule 2 — Selective Rollback: when the user wants a previous version as the base but wants to keep specific elements from the current version, the flow is: 1. Thinker Core identifies this as a selective rollback request 2. Clarification Agent asks one question if the user has not been specific enough: “Which specific parts from the current version do you want to keep?” 3. The previous version is restored as the base 4. Builder applies only the user-selected elements from the current version on top of that base 5. Reviewer checks the merged result

Rule 3 — Rollback never destroys history: whether full or selective, the act of rolling back does not delete any existing version. The version history always grows forward — it never loses entries. If a user rolls back to Version 2 and then makes a change, the history becomes: Version 1 → Version 2 → Version 3 (old) → Version 4 (new rollback-based version).

Rule 4 — Every selective rollback creates a new version: the merged result is saved as a brand new version entry, not an overwrite of any existing version. This means the user can always undo a selective rollback if the merge did not turn out as expected.

Rule 5 — User-selected changes always override rollback state: if there is any conflict between the restored base version and the elements the user chose to keep, the user’s explicit selection wins. Thinker Core never silently resolves a merge conflict — if a conflict is detected that cannot be automatically resolved, it is surfaced to the user with a clear description before Builder runs.

Selective Rollback flow summary:

```
 User: "The previous version was better, but keep these two things"
   ↓
Thinker Core: identifies as selective rollback
   ↓
Clarification (if needed): "Which two things do you want to keep?"
   ↓
```

```
Previous version restored as base (no Builder call)
  ↓
Builder applies only the selected elements
  ↓
Reviewer checks the merged result
  ↓
Saved as a new version
  ↓
User sees — rollback + targeted changes combined
```

Cost: selective rollback = one small Builder call (targeted apply only) + one Reviewer call. It counts as one small fix in the cost tier table (Section 9.5), not a full rebuild.

10. Technology Stack (V1 Recommendation)

Layer	Choice	Reasoning
Frontend	React + Vite	Matches existing developer experience
Backend	Node.js (Express) or Firebase Cloud Functions	Lightweight, matches existing Firebase familiarity
Database	Firestore	Document model fits the pipeline state object well
Auth	Firebase Auth	Reuse existing pattern
LLM Access	Direct API calls to each provider, made only from the backend, never the client	Protects API keys, matches the Resource Protection Law
Hosting	Firebase Hosting or Vercel	Simple, matches existing deployment habits

This stack choice is a recommendation, not a constitutional requirement. It is chosen because it overlaps heavily with tools already familiar, minimizing new tooling to learn for a V1 build.

11. Build Order (Implementation Roadmap)

This sequencing turns the constitutions' broader roadmap into concrete, shippable milestones.

Phase	Deliverable
Phase 1	Thinker Core skeleton plus the direct chat path (no agents yet, just passthrough chat)
Phase 2	Intent Agent and Clarification Agent (basic), with the hard gate between them
Phase 2.5	Smart Clarification Layer: Emotional Context Detection + Contradiction Detection + Assumption Surfacing + Progressive Disclosure + Intent Confidence Threshold (Section 5.2.2)
Phase 2.6	3-Level Deep Clarification + Signature Question Protocol (Sections 5.2.3, 5.2.5)
Phase 2.7	Constraint Discovery + Assumption Discovery + Alternative Discovery, with conditional triggers (Section 5.2.4)
Phase 3	Planner Agent and Builder Agent (single-step, no research yet)

Phase 4	Reviewer Agent and the one-retry loop back to Builder
Phase 5	Critic Agent and Judge Agent (Judge decides alone, no Consensus yet)
Phase 6	Consensus Agent and the borderline-routing logic from Judge
Phase 7	Research Agent (optional step augmentation)
Phase 8	Design Agent and the Builder/Design routing logic from Section 4.3
Phase 8.5	Strategy Agent — runs after Clarification, before Planner, on project requests only (Section 5.12)
Phase 9	Project memory persistence, long-term preference memory, and Decision Memory (Section 7.3)
Phase 10	Domain picker wiring: send selected domain from mobile UI to backend, and thread it through Intent/Clarification/Strategy/Planner/Builder per Section 4.4
Phase 10.5	Per-agent logging (Section 9.2) and post-delivery feedback loop (Section 9.4)
Phase 11	Polish: UI, error states, retry UX, and the launch checklist (Section 12)

Each phase should be independently testable before moving to the next.

12. V1 Launch Checklist

- Stable chat experience: the direct path works without errors, and every simple chat response reflects agreement (or flagged disagreement) between two models, never a single unverified model call (Section 6.3)
- Full agent pipeline completes end-to-end on at least three different project types (app, website, game)
- Strategy Agent produces a meaningful strategic brief for at least one project request, and that brief is visibly reflected in Planner’s output (e.g. MVP scope limits what Builder builds)
- Decision Memory saves an explicit user rule, applies it automatically to the next project without asking, and surfaces a conflict correctly when a new request contradicts a saved rule (Section 7.3)
- Design Agent produces a usable image for at least one project that requests visual assets, and that image passes through Reviewer correctly
- User can choose between describing their own style and letting Design Agent decide freely, and both paths produce a correctly-scoped image
- Approve / reject-and-regenerate / revise all work correctly for a generated image, and the 3-attempt cap is enforced
- Clarification gate cannot be bypassed
- Smart Clarification correctly detects emotional tone and adjusts question framing accordingly (Section 5.2.2)
- Contradiction Detection catches at least one case where a user’s later message contradicts an earlier constraint, and surfaces it explicitly rather than silently resolving it (Section 5.2.2)
- Signature Question fires on every project-type request; user response (or decline) is stored in Pipeline State and visible in per-agent log (Section 5.2.5)
- Intent Confidence Threshold correctly routes: high confidence proceeds, mid-confidence asks one confirmation, low confidence activates Goal Discovery Mode (Section 5.2.2)
- No credentials ever appear in stored memory (manual audit)
- Consensus Agent tested with intentionally borderline cases
- High-risk categories (auth, payment, data deletion, permissions) tested to confirm Consensus fires automatically, even when Reviewer and Critic both pass

- Token cost per request is logged and reviewed
- Final response is always a single merged answer, never raw agent dumps
- 5-stage output pipeline works end-to-end: user sees Thinking Summary → Blueprint → Live Preview → Approval → Final ZIP, and no stage proceeds without user confirmation (Section 16)
- Live Preview renders correctly for web projects within the app (Section 16.3)
- Language detection works correctly: user writes in any Tier 1 language and all agent responses, progress indicators, and final documentation are returned in that language (Section 17.3)
- RTL language (Arabic or Hebrew) renders correctly in the UI — text is right-aligned, layout is mirrored (Section 17.5)
- Generated code stays in English regardless of the user's language; README and documentation are in the user's language (Section 17.2)
- Per-agent log entries are written for every pipeline run (Section 9.2), and a failed run can be traced to a specific agent within minutes using those logs
- Post-delivery feedback works correctly for all three failure types: bug fix sends back to Builder, missing feature runs a partial Planner→Builder cycle, vague feedback asks one clarifying question before acting (Section 9.4)
- Cost-based fix limits enforced correctly: small fixes unlimited, medium fixes capped at 10, full rebuilds capped at 3 per project session (Section 9.5)
- Version history saves correctly on every Builder output, rollback restores a previous version without any Builder call, and the 10-version limit deletes oldest versions automatically (Section 9.6)
- Selective rollback works correctly: previous version restored as base, selected elements applied on top, result saved as new version, no existing version deleted (Section 9.7)
- Selecting a domain in the picker measurably changes pipeline output (verified by testing the same prompt with and without a domain selected and confirming the difference), per Section 4.4
- A full codebase and log search for provider/model names (Claude, Anthropic, GPT, OpenAI, Gemini, Google) confirms none are reachable from any user-facing surface, including error states (Section 6.5)

13. Explicitly Out of Scope for V1

Carried forward from the longer-term vision, not to be built now:

- Parallel multi-step Builder execution
- Automatic re-planning when a step fails repeatedly
- Native mobile app (the Play Store launch can wrap the web app initially)
- Organizational/team-shared memory
- Autonomous multi-day project system with no human checkpoint
- Native Thinker foundation model (V1 uses third-party model APIs only)
- 3D model generation (requires specialized mesh-generation tooling, not a simple API call like image generation)
- Animation and video generation (requires motion/timeline tooling well beyond Design Agent's V1 scope)
- UI/UX design as a distinct discipline from code: V1's Design Agent produces static images only, not interactive prototypes or design files (e.g. Figma-format output)

14. Open Decisions for Founder Review

These are choices this document made to keep V1 buildable. Flag any that should change before development starts:

1. ~~Consensus uses exactly three providers~~ — **Resolved:** confirmed for V1 (Section 6.6), with the model pool designed to be configurable beyond three in later versions.
2. Reviewer retry is capped at one loop — confirm this is acceptable or should be configurable.
3. Research Agent is optional and skippable per step — confirm this matches intended behavior, rather than "always research."
4. The technology stack assumes continuity with an existing Firebase-based project — confirm this is

desired versus a fresh stack.
5. The high-risk category list for mandatory Consensus (Section 8.2) currently covers authentication, payments, data deletion, and permissions — confirm this list is complete for your use case, or add categories specific to your projects.
6. Design Agent's image provider is not yet chosen between options like DALL-E or Stability AI — confirm which one based on cost, image quality for your use cases, and licensing terms for commercial use of generated images.

15. Goal Intelligence System

This section brings together the highest-level thinking layer of Thinker AI — the system that determines whether a user's goal is valid, well-formed, and worth building, before any technical work begins. This is what makes Thinker AI a thinker, not just a builder.

15.1 Goal Discovery

Goal Discovery is the process of understanding what the user actually wants to achieve — not what they literally typed. Many users arrive with vague intent: "I want to build something" or "I want to start a business." A standard AI tool would either ask for more details about a product that hasn't been defined yet, or worse, start building based on the surface-level request.

Thinker AI does neither. When a user's goal is unclear, Goal Discovery Mode activates (Section 5.2.1) and works through five layers of questions — one at a time, conversationally — to surface the real goal beneath the stated one. By the time Goal Discovery completes, Thinker AI knows not just what the user wants to build, but why, for whom, and what success means.

Goal Discovery is not an obstacle. It is the product. A user who spends 5 minutes in Goal Discovery before building is far more likely to end up with something useful than one who immediately gets a generic deliverable that misses the point.

15.2 Problem Discovery

Problem Discovery is a subset of Goal Discovery focused specifically on identifying the real problem being solved. This matters because many users describe a solution ("I want a social app") without being able to articulate the problem ("people in my niche community have nowhere to connect"). If the problem is not real, or not specific enough, no amount of good engineering will produce a successful product.

Thinker AI's Clarification Agent asks: "What specific problem will this project solve?" and follows up until the answer is specific. A vague problem ("people need to connect") gets a follow-up: "Who specifically, where, and why are they unable to connect right now?" A concrete problem ("Bangladeshi gamers have no Bengali-language Discord equivalent") is ready to proceed.

15.3 Idea Validation

After Goal Discovery and Problem Discovery, Strategy Agent runs an Idea Validation — a structured assessment of whether the user's idea, as stated, has a reasonable chance of achieving their goal. This is not a gatekeeping step (the user always decides whether to proceed), but a thinking tool that surfaces risks before they become expensive mistakes.

Idea Validation covers four questions:

1. **Is the problem real?** Does this problem actually exist, and are real people experiencing it?
2. **Is this idea the right solution?** Are there better, simpler, or cheaper ways to solve the same problem?
3. **Is this the right time?** Are there market, technology, or behavioral conditions that make this idea viable now?
4. **Why this team?** Does the user have a specific advantage — knowledge, access, or position — that

makes them the right person to build this?

The output of Idea Validation is not a go/no-go decision — it is a brief that Strategy Agent uses to sharpen the plan that Planner receives. If the idea has significant weaknesses, they are surfaced to the user explicitly, not silently worked around.

15.5 Founder Mode

Founder Mode is a special mode activated when a user is not just building a feature but thinking through an entire product, business, or startup idea. It can be triggered explicitly ("Activate Founder Mode") or detected automatically when the user's request involves market, revenue, competition, or business model language.

What Founder Mode changes:

In normal mode, Strategy Agent produces a brief focused on the technical project. In Founder Mode, Strategy Agent expands its scope to cover the full business context:

Normal Mode	Founder Mode
Goal clarity for this build	Problem → Market → Competition → Business model
MVP scope (features)	MVP scope (minimum to validate the business, not just the feature)
Key technical risks	Full risk landscape: market, technical, financial, execution
Success criteria for the deliverable	Success criteria for the business itself

Founder Mode output — presented to user before any building begins:

- **Problem Statement:** one clear sentence describing the problem being solved
- **Market Assessment:** who has this problem, how many of them, what do they use today
- **Competitive Differentiation:** why this idea, why now, why this team
- **MVP Recommendation:** the smallest thing that could validate the core assumption
- **Go / Caution / No-Go:** Thinker AI's honest assessment, with reasoning

Go / Caution / No-Go is always advisory, never a block. The user proceeds on their own judgment. Thinker AI's role is to make sure the judgment is informed.

16. Output Pipeline — Preview Before Build

This section defines how Thinker AI delivers a project to the user. Instead of building everything silently and delivering a ZIP file at the end, Thinker AI follows a 5-stage pipeline where the user sees and approves each stage before the next begins. This is the implementation of the Preview Before Build Law (Section 3).

16.1 The Five Stages

Stage 1 — Thinking Summary (user sees this first)

Before anything is built or planned in detail, Thinker AI shows the user what it understood and what it is planning to do. This is the output of Clarification Agent + Strategy Agent, presented clearly.

User sees: - **Goal:** what Thinker AI understood the user wants to achieve (in outcome terms, not feature terms) - **Strategy brief:** key risks, MVP scope, success criteria (from Strategy Agent) - **Proposed approach:** in plain language, not technical jargon

User action: Review and confirm, or correct misunderstandings before anything is built. This is the cheapest point to catch a wrong direction — no Builder tokens have been spent yet.

Stage 2 — Blueprint Preview

After Stage 1 approval, Planner Agent runs and produces a structured blueprint. The user sees the full plan before Builder writes a single line of code.

User sees: - List of screens / sections / components that will be built - Brief description of each (e.g. “Home Screen — restaurant list with search and filter”) - Tech stack that will be used (Frontend, Backend, Database) - Estimated complexity and any flagged risks

User action: Approve the blueprint, remove or add items, or ask for changes. Changes here cost almost nothing — Planner re-runs, no Builder call needed.

Stage 3 — Build Preview (live progress)

After Stage 2 approval, Builder Agent runs screen by screen. The user sees real-time progress and gets one approval checkpoint — after the first screen — before the rest builds automatically.

Left side — Streaming progress indicator:

```
✓ Requirements Complete
✓ Strategy Complete
✓ Blueprint Approved
⧖ Building Home Screen...
⧖ Building Chat Screen
⧖ Building Profile Screen
⧖ Building Settings Screen
```

Each item updates live as Builder completes it. The user always knows what is happening and what comes next.

Right side — Live Preview (updates as each screen completes):

As Builder finishes each screen, it appears immediately in the preview panel — not as code, but as the actual rendered UI. The user can swipe or tap between completed screens while Builder continues working on the remaining ones.

First-screen approval checkpoint (the only mandatory pause during build):

When the first screen is complete, Builder pauses and Thinker Core asks:

> “Home Screen is ready. Does the style, layout, and feel match what you had in mind? I’ll continue building the remaining screens based on this design direction.
>
> [✅ Looks good, continue] [🔄 Change something] [↩ Start over]”

- ✅ **Looks good:** Builder continues with all remaining screens automatically, following the same design direction. No more pauses until Stage 4.
- 🔄 **Change something:** User describes what to change (e.g. “make the header larger”, “use dark theme”). This counts as one small fix (Section 9.5). Builder updates only the first screen, then asks for approval again before continuing.

Why only the first screen: the first screen establishes the visual language — colors, typography, spacing, component style — that all subsequent screens follow. If the first screen is right, the rest will be right. Asking for approval after every screen would be slow without adding much value; asking only once at the start gives the user meaningful control at the lowest cost.

When all screens are complete: the full app is visible in the preview panel. User can swipe through all screens (Home → Chat → Profile → Settings) before moving to Stage 4.

Stage 4 — Approval

After seeing the live preview, the user decides:

User choice	What happens
✅ Looks good, continue	Project is finalized. Final output package is prepared.
🔄 Change something specific	Post-delivery feedback loop activates (Section 9.4). Targeted fix only, not a full rebuild.
↩ Go back to blueprint	Version History allows rollback to Stage 2 state. Blueprint can be revised without rebuilding.

No output is finalized until the user explicitly approves. Thinker Core never auto-finalizes.

Stage 5 — Final Output

After user approval, Thinker Core prepares the final deliverable package:

What the user receives: - **Project Summary** — goal, strategy, what was built, key decisions made - **Live Preview Link** — permanent (not temporary) hosted version, if applicable - **Download Package (ZIP)** — all source files, organized by folder structure, with a README explaining how to run or deploy - **Architecture Notes** — tech stack used, how the pieces connect, how to extend it later

ZIP structure (standard across all project types):

```
project-name/
  README.md          ← How to run, deploy, and extend
  frontend/          ← All UI code
  backend/           ← All server/API code
  database/          ← Schema and seed files
  assets/            ← Images, icons, generated by Design Agent
  docs/              ← Architecture notes, API docs
```

16.2 What Changes for Simple Requests

Not every request needs all 5 stages. The pipeline scales down for simpler requests:

Request complexity	Stages shown
Simple single-file output (e.g. a script, a single page)	Stage 1 (brief) → Stage 3 (build) → Stage 5 (file download). Stages 2 and 4 skipped.
Medium project (2–5 components)	All 5 stages, but Stage 2 blueprint is short
Large project (many screens/components)	Full 5 stages with detailed Stage 2 blueprint

Rule: the user is never surprised by what was built. If the output is too simple to need a full blueprint stage, Thinker Core still shows a one-line summary of what it is about to build and waits for a brief confirmation before running Builder.

16.3 Live Preview — Technical Note

The Live Preview in Stage 3 and Stage 5 requires a temporary hosting environment. In V1:

- **Web projects (HTML/CSS/JS, React):** rendered in an iframe or sandboxed browser view within the app itself — no external hosting needed for preview
- **Backend/API projects:** a summary of endpoints and expected behavior is shown; live API testing is a V2 feature
- **Mobile projects:** a web-based preview of the UI is shown; actual APK/IPA generation is out of scope for V1 (see Section 13)

17. Multi-Language Support

Thinker AI supports conversations and output in 100+ languages. This section defines how language support works across the entire pipeline — from the user's first message to the final delivered code and documentation.

17.1 Core Principle

Thinker AI detects the user's language automatically from their first message and responds in that same language throughout the entire session — all agent responses, clarification questions, strategy briefs, progress indicators, error messages, and final summaries. The user never has to specify their language or change a setting.

17.2 What Gets Translated vs. What Stays in English

Not everything in a software project should be translated. The rule is clear:

Content Type	Language Used	Reason
All conversation with the user	User's language	This is the product interface
Clarification questions	User's language	User must understand these clearly
Strategy brief, Goal Discovery	User's language	Thinking happens in the user's language
Progress indicators and status messages	User's language	UX must be native-feeling
Error messages shown to user	User's language	Must be understandable
Generated code (variables, functions, classes)	English	Industry standard; code must be readable by any developer
Code comments	User's language (if requested) or English (default)	User's choice via Decision Memory
README and documentation	User's language	User needs to understand their own project
Folder and file names	English	Compatibility and portability
API endpoint names	English	Industry standard

17.3 Language Detection

Language detection happens at the Intent Agent level — the first LLM call in every pipeline. Intent Agent outputs both the intent type and the detected language code (e.g. `en`, `bn`, `ar`, `hi`, `zh`, `es`, `fr`). This language code is passed to every subsequent agent in the pipeline state object, so all agents respond in the correct language without each needing to detect it independently.

If a user switches languages mid-conversation (e.g. starts in English, then writes in French), Thinker Core detects the change and updates the language code from that message forward. Previous messages are not re-translated.

17.4 Supported Language Tiers

Not all 100+ languages have equal LLM support quality. V1 acknowledges three tiers:

Tier	Languages	Quality in V1
Tier 1 — Full support	English, French, Spanish, German,	High quality across all agents

	Portuguese, Italian, Dutch, Japanese, Korean, Chinese (Simplified), Arabic, Hindi, Bengali, Turkish, Russian, Polish, Vietnamese, Indonesian	
Tier 2 — Good support	50+ additional languages including Urdu, Malay, Thai, Swahili, Romanian, Czech, Swedish, Norwegian, Danish, Finnish, Greek, Hebrew, and others	Good quality; occasional phrasing issues in complex reasoning
Tier 3 — Basic support	Remaining languages supported by the underlying model providers	Response quality depends on the model's training; user is informed if quality may be limited

Thinker AI does not block any language — it serves all languages the underlying models support, but is honest about quality differences where they exist.

17.5 Right-to-Left Language Support

Languages written right-to-left (Arabic, Hebrew, Urdu, Farsi, and others) are fully supported in all user-facing text. The UI must render these correctly — right-aligned text, mirrored layout where appropriate. This is a UI implementation requirement, not just a translation requirement. V1 must be tested with at least one RTL language before launch.

17.6 Language and Decision Memory

A user can save language preferences to Decision Memory (Section 7.3):

- "Always respond in English even if I write in another language"
- "Always write code comments in Spanish"
- "Use formal language" / "Use casual language"

These are stored as decision memory entries and applied automatically to all future sessions.

17.7 Language Support in Output Pipeline

The 5-stage output pipeline (Section 16) is fully localized:

- Stage 1 Thinking Summary — in user's language
- Stage 2 Blueprint — screen names and descriptions in user's language; tech stack names in English
- Stage 3 Build Preview — progress indicators in user's language; code itself in English
- Stage 4 Approval prompts — in user's language
- Stage 5 README and documentation — in user's language; code and file names in English

18. Business Model & Credit System

This section defines Thinker AI's pricing structure, credit economy, thinking levels, and how all three connect to the agent pipeline and Goal Discovery system.

18.1 Plans

Feature	Free Trial	Pro	Founder
Price	Free	$19/month	$59/month
Annual	—	$199/year	$589/year
Think Credits	50 (one-time)	1,500/month	5,000/month
Goal Discovery	Limited	Full	Full

Strategy Agent	✅	✅	✅
Project Planning	❌	✅	✅
Project Memory	❌	✅	✅
Decision Memory	❌	❌	✅
Research Agent	❌	Limited	Unlimited
Consensus Thinking	❌	❌	Full
Large Projects	❌	Medium	Unlimited
Priority Queue	❌	❌	✅

18.2 Thinking Levels

Every request is classified into one of four thinking levels. The level determines how deeply Thinker AI thinks before responding, how many clarification questions it asks, and how many credits are consumed.

Level	Purpose	Credit Cost
⚡ Low Thinking	Fast answers — simple chat, factual questions (dual-model verified, Section 6.3)	2
🧠 Medium Thinking	Analysis — comparisons, recommendations, moderate planning	3
🚀 High Thinking	Deep planning — full project pipeline, Goal Discovery, Strategy	10
👥 Consensus Thinking	Multi-model validation — Assumption Discovery + Strategy + Judge + Consensus Agent	30

Thinker AI Golden Rule:

> More Thinking = More Questions = More Validation = More Credits

The user pays not for answers, but for depth of thought. This is what makes the credit system honest — a user who wants a quick answer pays 1 credit; a user who wants their startup idea fully validated before building pays 30. The value delivered scales with the cost.

18.3 Clarification Depth by Thinking Level

The thinking level directly determines how deeply Clarification Agent probes before proceeding.

Thinking Level	Clarification Style	Max Questions
⚡ Low	Direct — minimal friction	0–2
🧠 Medium	Smart — adaptive questions (Section 5.2.2)	3–5
🚀 High	Deep Discovery — full 3-Level Deep Clarification (Section 5.2.3)	5–10
👥 Consensus	Deep + Challenge + Validation — all layers including Assumption Discovery and Signature Question	Dynamic

18.4 Goal Discovery Behavior by Thinking Level

Discovery Layer	Low	Medium	High	Consensus
Goal Discovery	✗	Light	Full	Full
Problem Discovery	✗	Light	Full	Full
Audience Discovery	✗	Light	Full	Full
MVP Discovery	✗	Optional	Full	Full
Success Discovery	✗	Optional	Full	Full
Assumption Discovery	✗	✗	Conditional	Full
Constraint Discovery	✗	Optional	Full	Full
Alternative Discovery	✗	✗	Conditional	Full
Reality Challenge	✗	✗	Light	Strong

18.5 Credit Cost Table

Action	Credits
Simple Chat (dual-model verified, Section 6.3)	2
Clarification Reply	2
Research Step	3
Medium Analysis	4
Image Generation	10
Deep Think Session	10
Consensus Session	25
Small Fix	1
Medium Fix	5
Full Rebuild	50
Rollback	0

Note: Simple Chat costs 2 credits, not 1, because every chat message — even the simplest factual question — is sent to two models in parallel and verified before answering (Section 6.3). This is the new baseline; there is no cheaper unverified tier. Rollback costs 0 credits — it is a database lookup with no agent calls (Section 9.6). Full Rebuild costs 50 credits because it runs the entire pipeline from Planner onwards — this aligns with the Resource Protection Law (Section 6.2) and gives users a strong incentive to clarify requirements before building rather than rebuilding repeatedly.

18.6 Thinking Level Auto-Detection

Thinker Core automatically selects the thinking level based on Intent Agent's classification. The user can override this selection before proceeding.

Intent Type	Default Thinking Level
Simple factual question	⚡ Low
Comparison or recommendation	🧠 Medium
Project build request (app/website/game)	🚀 High

Startup validation / idea stress-test	👥 Consensus
Follow-up on existing project	Same level as original request

Plan restriction: Consensus Thinking is only available on the Founder plan. If a Free or Pro user's request is classified as Consensus-level, Thinker Core defaults to High Thinking and notifies the user that deeper validation is available on the Founder plan.

18.7 Credit Exhaustion Behavior

When a user's credits run low, Thinker Core behaves conservatively:

- At 20% credits remaining: user is notified at the start of each session
- At 5% credits remaining: Thinker Core switches all requests to Low Thinking automatically, notifies the user, and asks if they want to upgrade before starting any new project
- At 0% credits: only Simple Chat (2 credits) is available; all pipeline requests are paused until credits are renewed

Credits never expire mid-session. If a user starts a project with enough credits and runs out during execution, the current pipeline run completes. Credits are deducted only after a stage is successfully completed, never in advance.

19. Implemented Features — Codebase Reference

This section documents what has actually been built in the codebase as of the June 2026 review. It bridges the spec (what should be built) with the implementation (what has been built), so any developer joining the project can see the current state at a glance.

19.1 New Agents Implemented

Beyond the original nine constitutional agents, three additional agents are now in the codebase:

Design Agent (`designAgent.ts`): Generates image prompts via LLM then calls DALL-E 3 (`dall-e-3`, 1024×1024, standard quality) via the OpenAI API. Falls back to a placeholder status if no API key is present. Accepts `stepDescription`, `styleHints`, and `brandRequirements` as input. Returns `imageUrl`, `imagePrompt`, `description`, and a `status` field (`generated` | `placeholder` | `failed`).

Strategy Agent (`strategyAgent.ts`): Produces a structured strategic brief before Planner runs. Accepts `signatureQuestionResponse` and `constraintFindings` from Clarification Agent as inputs alongside the goal and requirements. Outputs `goalClarity`, `mvpScope`, `keyRisks`, `successCriteria`, `ideaValidation` (with `assessment`: go / caution / no-go), `founderMode` flag, and `strategicBrief` string. Auto-activates Founder Mode when market, revenue, competition, or business model language is detected.

Document Agent (`documentAgent.ts`): The 13th agent — handles document upload and analysis. Extracts `summary`, `key_points`, `sections`, `data_tables`, `entities`, detected `language`, `page_count`, and `word_count` from any uploaded document. Responds in the document's own language. Triggers a `document_analysis` pipeline status.

19.2 Clarification Agent — Smart Mode Implementation

`clarificationAgent.ts` has been significantly expanded from a simple requirement checklist to a full Smart Clarification system. Key implementation details:

- **Signature Question** is defined as a constant: *"Why do you believe this is the right solution to your problem?"* — fires on every non-chat request unless the user has already answered it or says "just build it"

- **Intent Confidence Threshold** is enforced in code: if `intentConfidence < 70`, Goal Discovery questions are automatically injected regardless of what the LLM returned
- **"Just build it" detection** via regex (`/just build|don't ask|no questions|proceed|go ahead/i`) — bypasses all clarification and sets `complete: true` immediately
- **Thinking Level** controls max question count: Low=2, Medium=5, High/Consensus=10
- **Heuristic fallback** (`heuristicClarify`) runs if the LLM call fails — guarantees the agent never crashes the pipeline
- **Goal Discovery questions** are hard-coded for High/Consensus levels with specific layer labels: `goal`, `problem`, `audience`, `mvp`, `success`

19.3 Intent Agent — Language Detection

`intentAgent.ts` now outputs a `detectedLanguage` field (ISO 639-1 code) on every classification. Language detection uses two approaches:

1. **LLM detection** (primary): the classification prompt explicitly asks for the language code based on the message text
2. **Unicode heuristic** (fallback): character range checks for Bengali (0980–09FF), Arabic (0600–06FF), Chinese (4E00–9FFF), Japanese (3040–30FF), Korean (AC00–D7AF), Cyrillic (0400–04FF), Devanagari (0900–097F), Hebrew (0590–05FF), Thai (0E00–0E7F)

25 languages are explicitly named in the `LANGUAGE_NAMES` map. The detected language code travels through `PipelineState.detectedLanguage` and is emitted as a `language_detected` event to the mobile client.

19.4 New API Routes

Route	Purpose
`POST /api/feedback`	Post-delivery feedback loop (Section 9.4) — classifies feedback as bug/missing_feature/style/vague and returns fix type, suggested prompt, and credit estimate
`GET /api/failover-log`	Returns per-session failover log for Founder plan debugging
`GET/POST /api/billing/plans`	Lists Stripe plans with prices
`POST /api/billing/checkout`	Creates a Stripe Checkout session
`POST /api/billing/portal`	Opens Stripe Customer Portal for subscription management
`POST /api/billing/webhook`	Handles Stripe webhook events (payment success, subscription changes)
`POST /api/upload`	File/document upload endpoint, feeds into Document Agent
`GET /api/download/:id`	Generates and serves a ZIP file of Builder output (Section 16, Stage 5)

19.5 Pipeline State — New Fields

`PipelineState` (in `types/pipeline.ts`) has been expanded with the following fields beyond what the original spec described:

Field	Type	Purpose
`thinkingLevel`	ThinkingLevel	Low/Medium/High/Consensus — drives Clarification depth and agent routing
`planTier`	PlanTier	free/pro/founder/enterprise — gates feature access

`detectedLanguage`	string	ISO 639-1 code detected by Intent Agent
`signatureQuestionResponse`	string | null	User's answer to the Signature Question
`signatureQuestionAnswered`	boolean	Whether the question has been answered or skipped
`constraintFindings`	ConstraintFindings	Time/budget/skill/network/assets from Clarification
`assumptionFlags`	AssumptionFlag[]	Surfaced assumptions with confirmed/uncertain/flagged status
`clarificationDepth`	number	How many clarification layers ran
`clarificationLayers`	string[]	Which specific layers ran
`goalDiscoveryMode`	boolean	Whether Goal Discovery Mode was activated
`decisionMemory`	DecisionMemoryEntry[]	Saved user rules for this session
`blueprintApproved`	boolean	Whether user approved the Blueprint (Stage 2)
`version_history`	VersionSnapshot[]	All Builder outputs saved by version number
`current_version`	number	Current version number
`medium_fix_count`	number	Medium fixes used (limit: 10)
`full_rebuild_count`	number	Full rebuilds used (limit: 3)
`routing_history`	RoutingHistoryEntry[]	Full trace of which agent ran and what it decided
`failover_log`	FailoverLogEntry[]	Log of model failovers (agent, failed model, replacement)
`thinkCreditsUsed`	number	Credits consumed in this pipeline run
`agentLogs`	AgentLog[]	Per-agent structured logs (Section 9.2)

New Pipeline Statuses: `blueprint_review`, `document_analysis`, `awaiting_signature`, `halted` — beyond the original set.

19.6 Pipeline Events — New Event Types

The streaming event system (`PipelineEvent`) has been extended with new event types that the mobile client listens for:

Event	When it fires	What mobile does
`signature_question`	Clarification decides Signature Question is needed	Shows `SignatureQuestionCard`
`thinking_summary`	After Strategy Agent completes	Shows `ThinkingSummaryCard` (Stage 1)
`blueprint_ready`	After Planner completes	Shows `BlueprintApprovalCard` (Stage 2)
`decision_saved`	When a user decision rule is detected	Shows `DecisionMemoryBanner`
`version_saved`	Each time Builder output is saved	Updates version counter
`language_detected`	After Intent Agent runs	Updates UI language hint

`document_ready`	After Document Agent completes	Shows document analysis result
`credit_confirm`	Before high-cost pipeline action	Shows `CreditConfirmModal`
`approval_needed`	After full pipeline completes	Shows approval UI (Stage 4)
`image_approval_needed`	After Design Agent generates an image	Shows `ImageOutputCard`
`feedback_prompt`	After user receives output	Shows post-delivery feedback options
`fix_limit_reached`	When medium or rebuild limit is hit	Shows `FixLimitModal`
`founder_mode_activated`	When Strategy Agent sets founderMode=true	Shows Founder Mode banner
`pipeline_halt`	When pipeline is paused	Shows `HaltCard`
`failover`	When a model is swapped due to failure	Shows brief failover notice
`final_output`	Stage 5 complete	Shows `FinalOutputCard`

19.7 Mobile UI — New Screens

Screen	File	Purpose
Onboarding	`app/onboarding.tsx`	Multi-slide onboarding with plan picker — shown once on first launch
Projects	`app/projects.tsx`	Project list with filter (all/active/completed/pinned), search, and status indicators
Voice	`app/voice.tsx`	Animated voice input screen with waveform bars and haptic feedback
Profile	`app/profile.tsx`	User profile, plan info, and settings shortcut
Settings	`app/settings.tsx`	8-tab settings: General, Pipeline, Memory, Files, Credits, Notifications, Privacy, Advanced

19.8 Mobile UI — New Components

Component	Purpose
`SignatureQuestionCard`	Shows the Signature Question with text input, Skip, and Proceed buttons
`ThinkingSummaryCard`	Stage 1 — shows goal understanding, strategy brief, thinking level, and credit cost
`BlueprintApprovalCard`	Stage 2 — shows plan steps with output type icons, tech stack, Approve/Modify/Start Over
`CreditConfirmModal`	Pre-action credit confirmation with level label, cost, and balance
`DecisionMemoryBanner`	Animated banner shown when a user decision rule is detected and saved
`FixCounterBar`	Visual progress bar showing medium fix count (0/10) and rebuild count (0/3)
`FixLimitModal`	Modal shown when a fix limit is reached
`FinalOutputCard`	Stage 5 — shows credits used, agent count, duration, Live

	Preview link, and ZIP download
`ImageOutputCard`	Image approval with Approve/Revise/Regenerate, attempt counter (e.g. 1/3)
`HaltCard`	Pipeline pause notice with completed/total steps, reason, and Resume/Rescope/Dismiss
`RightPanel`	Right-side sliding panel for additional context/settings
`ThinkerLogo`	Animated Thinker AI logo component
`ErrorBoundary` / `ErrorFallback`	React error boundary with graceful fallback UI
`KeyboardAwareScrollViewCompat`	Cross-platform keyboard-aware scroll wrapper

19.9 Local Persistence — AppContext

Conversations (including messages, agent type, detected language, decision memory, and fix counters) are persisted to `AsyncStorage` under the key `@thinkai_conversations_v2`. This means conversation history survives app restarts on the device. This is client-side persistence only — it is not synced to the backend database (which has an empty schema as of this review).

19.10 Known Implementation Gaps (as of June 2026 review)

These items are specified in the architecture but not yet implemented:

Gap	Spec Reference	Impact
Database schema is empty — no server-side persistence	Sections 7.2, 7.3, 9.2	Project Memory, Decision Memory, and Version History are lost on server restart
Credit deduction logic not implemented	Section 18	`thinkCreditsUsed` is tracked but never deducted from a balance
Version History save not triggered in Builder	Section 9.6	Rollback and Selective Rollback cannot function
Multi-language response threading incomplete	Section 17	Language is detected but not passed to agent prompts
Live Preview (Stage 3) not implemented	Section 16.1	Users see blueprint then final output, skipping the first-screen approval checkpoint

End of V1 Technical Architecture Specification.